DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SHARDA SCHOOL OF COMPUTER SCIENCE & ENGINEERING

SHARDA UNIVERSITY, GREATER NOIDA

Development of an Agentic-RAG Powered Research Assistant for Academia

A project submitted
in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering

by

Basharat Hassan (2022521926)

Supervised by:

Mr. Mohammad Asim, Assistant Professor

DECEMBER, 2025

CERTIFICATE

This is to certify that the report entitled "Development of an Agentic RAG-Powered Research Assistant for Academia" submitted by Basharat Hassan (2022521926) to Sharda University, towards the fulfilment of requirements of the degree of "Bachelor of Technology" is a record of Bonafide final year Project work carried out by them in the "Department of Computer Science & Engineering, Sharda School of Computer Science and Engineering, Sharda University". The results/findings contained in this Project have not been submitted in part or full to any other University/Institute forward of any other Degree/Diploma.

Name: Dr. Sudeep Varshney
Place: Sharda University
Date:

Name: Mr. Mohammad Asim
Designation: Assistant Professor

Signature of External Examiner
Date:

ACKNOWLEDGEMENT

A major project is a golden opportunity for learning and self-development. We consider ourselves very lucky and honoured to have so many wonderful people lead us through the completion of this project. First and foremost, we would like to thank Dr. Sudeep Varshney, HOD, CSE who gave us an opportunity to undertake this project. Our grateful thanks to Mr. Mohammad Asim for his guidance in our project work. Mr. Mohammad Asim, who despite being extraordinarily busy with academics, took a timeout to hear, guide and keep us on the correct path. We do not know where we would have been without his help. CSE department monitored our progress and arranged all facilities to make life easier. We choose this moment to acknowledge their contribution gratefully.

Basharat Hassan (202251926)

ABSTRACT

The system is geared towards addressing real time issue of locating accurate, structured and citation-based information in ways that are not subject to delays and inconsistencies of manual research. The real-world users be it students, researchers or professionals are overwhelmed by information overload, unreliable sources and time to process raw data into meaningful knowledge in most real-world situations. To mitigate this the platform combines an Agentic Retrieval-Augmented Generation (RAG) pipeline using Groq to process high-speed and accurately retrieve data. The user begins by creating an account, log-in and profile management. It is a system that, using the dashboard, one only needs to key in a topic, and the system will automatically narrow the query and access quality content in the relevant sources. The agentic backend system refinances the information and filters and summarizes it to create an articulate review with precise references. This automated pipeline also saves the manual search, cross-verification and summarization processes, thus saving much effort and enhancing reliability. The end result is well-formatted, and it has the opportunity to export the content to be used in academics or professionally. The system offers a quick, reliable, and effectively structured review by integrating automated retrieval, intelligent summary, and user friendly-interaction as a means of offering an efficient real-time solution to users that require efficient, reliable, and fast reviews.

Contents

LIST OF TABLES I
LIST OF FIGURES II
LIST OF ABBREVIATION III
CHAPTER 1 INTRODUCTION 1
1.1 Background and Motivation 2
1.2 Problem Definition 2
1.3 Research Objectives 3
1.4 Scope of the Project 3
1.5 Key Contributions 4
1.6 Organization of the Project Report 4
CHAPTER 2 LITERATURE REVIEW & THEORITICAL FOUNDATION 5
2.1 Introduction to Retrieval-Augmented Generation 6
2.2 Agentic AI Systems 6
2.3 AI-Based Research Assistants 7
2.4 Related Work 7
2.5 Analysis of Existing Academic Search Platforms 9
2.6 Identified Research Gaps 10
2.7 Summary of Literature Findings 10
CHAPTER 3: SYSTEM REQUIREMENTS & FEASIBILITY ANALYSIS 11
3.1 Functional Requirements 12
3.2 Non-Functional Requirements 13
3.3 Feasibility Study 13
3.4 Chapter Summary 15
CHAPTER 4: SYSTEM ARCHITECTURE & DESIGN 16
4.1 High-Level System Overview 17
4.2 Overall Project Workflow 17
18
4.3 System Architecture 18
4.4 Module-Wise Design 19
4.4.1 User Management Module 19
4.4.2 Query Refinement Module 19

4.4.3 Retrieval Module 19
4.4.4 Vector Store Layer (FAISS & Chroma) 19
4.4.5 Agentic RAG Controller 19
4.4.6 Summarization & Citation Module 19
4.5 Data Flow Design 20
4.6 UML Diagrams 20
4.6.1 Sequence Diagram 20
4.6.2 Activity Diagram 21
4.7 Deployment Architecture 21
4.8 Design Decisions and Trade-offs 22
CHAPTER 5: IMPLEMENTATION DETAILS & RESULTS 23
5.1 Introduction 24
5.2 Technological Stack Overview 24
5.3 Backend Implementation 25
5.4 Agentic RAG Pipeline Implementation 25
5.5 Vector Database Implementation 26
5.6 Frontend Implementation 26
5.7 User Authentication and Access Interface 27
5.8 Dashboard and Research Search Interface 28
5.9 Research Paper Retrieval and Viewing 29
5.10 AI-Based Summary Generation 30
5.11 Output Results for Different Search Domains 30
5.12 Summary of Results 31
CHAPTER 6: DISCUSSION 32
6.1 System Strengths 33
6.2 Limitations of the Current System 33
6.3 Ethical and Academic Integrity Considerations 34
CHAPTER 7: CONCLUSION 35
5.1 Conclusion 36
5.2 Future Scope 36
REFERENCES 38
ANNEXURE - I 39

Sample System Outputs.....39
ANNEXURE – II 40

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Literature Review	8
Table 3.1	Functional Requirements	12
Table 3.2	Non-Functional Requirements	13
Table 3.3	Types of Feasibility	14

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 4.1	Overall Project Workflow Diagram	18
Figure 4.2	System Architecture Diagram	18
Figure 4.3	Level 0 Data Flow Diagram (DFD)	20
Figure 4.4	Sequence Diagram	20
Figure 4.5	Activity Diagram	21
Figure 4.6	Deployment Architecture Diagram	21
Figure 5.1	Clean login page for ScholarCite platform with email and password fields.	27
Figure 5.2	ScholarCite dashboard with research paper search bar and feature highlights	28
Figure 5.3	Academic research platform with search filters and paper database	28
Figure 5.4	Research paper summary page with abstract and AI-generated insights	29
Figure 5.5	Redirection to original research paper PDF source	29
Figure 5.6	AI summary generation page for academic research paper	30
Figure 5.7	Output results for biology-based academic search	31

LIST OF ABBREVIATION

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BibTeX	Bibliographic Text Format
DB	Database
DFD	Data Flow Diagram
DOI	Digital Object Identifier
FAISS	Facebook AI Similarity Search
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
LLM	Large Language Model
NLP	Natural Language Processing
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
JSON	JavaScript Object Notation

CHAPTER 1 INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The rapid growth of digital academic content form has revolutionized the research process. The digital works of scholarly literature are accessible today in large repositories that allow students, researchers, and professionals to access them via online platforms and through free-of-charge archives. Although such availability has reduced the obstacles in accessing information, it has also brought the new challenges regarding information overload, relevance evaluation and effective knowledge extraction.

The current academic search engines normally give numerous results to a single query. Users have to scan in titles manually, abstracts, and authenticate credibility, contrast between various papers, and integrate findings into logical insights. This is a time-intensive, cognitively challenging process that is likely to be inconsistent, especially in relation to interdisciplinary or novel research subjects. This has led to wastage of precious time in information search and filtering instead of thinking and analyzing information.

The new developments in Artificial Intelligence, specifically the large language models and Retrieval-Augmented Generation (RAG) demonstrate high potential in supporting knowledge-intensive tasks. The vast majority of current AI-based tools are however standalone summarizers or use fixed retrieval pipelines, which restricts the reasoning abilities of them on many steps of the research process. It drives the desire to have a smart, agent-based research assistant that is capable of retrieving, processing and synthesizing academic data in a systematic and trustworthy way independently.

1.2 Problem Definition

Although there are now digital libraries and other academic search system available, it is not an easy task to carry out good literature research. The classical search systems that are based on the use of keywords cannot easily grasp what the user is trying to achieve through the search so the search results become either too general or too narrow. Therefore, users have to rephrase search queries several times, they have to rate the sources by hand and they have to integrate information within several documents. Also, there is minimal support of structured summarization and citation-

aware content generation in the existing systems. Researchers have to find major insights manually and make sure that the content generated is based on original sources. Such a manual workflow does not only prolong the working effort but also creates a possibility of missing a crucial detail or presenting scholarly results in a biased way.

The main issue of this project is that there is no coherent, smart research assistant knowing user intent, finding semantically relevant academic information and producing structured, citation-based outputs in real time. The solution is that a system is required to minimize the use of manual intervention and retain academic integrity and transparency.

1.3 Research Objectives

The main goal of the project consists in designing and developing an intelligent research assistant that facilitates exploration of academic literature with the help of an Agentic Retrieval-Augmented Generation process. The particular goals are:

- To build a system that would be able to retrieve the relevant academic papers in relation to user queries semantically.
- To develop an agent-based pipeline that can coordinate several research activities including query refinement, query retrieval, query summarization, and citation generation.
- To combine a context-scrutinizing document matching mechanism based on vectors.
- To produce summaries that are concise and structured and based on retrieved academic sources.
- To offer interactive and user-friendly web interface in academic research support.

1.4 Scope of the Project

The scope of this project is the creation of a web-based research assistant that deals with the retrieval and summarization of academic literature. The system is run with arXiv as the only source of research papers in it so that it is guaranteed that the quality of the available open-access scholarly information is high.

The project is favourable to text-based research articles and not multimodal (images, tables, and mathematical representation of text) documents. This work does not involve any custom machine learning model training; instead, the system uses pre-trained models entirely to perform inference, and uses the combination of the vector databases to perform the retrieval.

The platform will serve as a support resource in research instead of a substitute of the human judgment, allowing users to search through literature more effectively without the need to lose access to primary sources.

1.5 Key Contributions

- **Agentic RAG-Based Research Pipeline:** The design presents an agent-based Retrieval-Augmented Generation workflow to coordinate the various research processes in a systematic process.
- **Semantic Retrieval Using Vector Databases:** With the help of embedding-based similarity search with FAISS and ChromaDB, the system allows to conduct context-aware retrieval in addition to traditional key-word matching.
- **Interactive Academic Research Assistant:** The site offers a user-friendly interface, enabling users to search by research topic, explore retrieved publications, create summaries that are generated with the help of AI, and respond to the system in real time.

All of these contributions would serve to illustrate how agentic AI architectures and RAG architectures can be successfully implemented to academic research processes.

1.6 Organization of the Project Report

In this project report, eight chapters are presented.

Chapter 1 presents the background, motivation, project objectives and scope of the project.

Chapter 2 includes an elaborate review of the literature and explicates the available studies within the area of Retrieval-Augmented Generation and agentic AI systems.

Chapter 3 presents the feasibility analysis and the system requirements.

Chapter 4 gives a detailed description of the system architecture and design.

Chapter 5 shows how the system was implemented and what technologies were applied.

Chapter 6, the results and outputs of the system are given.

Chapter 7 will address the strengths, limitations and ethical issues of the proposed system.

Chapter 8 will be the final chapter of the Project Report, and it will describe possible ways of working in the future.

CHAPTER 2 LITERATURE REVIEW & THEORITICAL FOUNDATION

CHAPTER 2

LITERATURE REVIEW & THEORETICAL FOUNDATION

2.1 Introduction to Retrieval-Augmented Generation

This chapter is aimed at researching the existing studies and systems connected with information retrieval, academic research assistance, and Retrieval-Augmented Generation (RAG). The literature survey assists in knowing how research related challenges are being solved using the current technologies and the areas of limitations that still exist. This chapter addresses gaps in the analysis of the past work that will encourage the creation of the proposed system.

Recent studies have further developed this paradigm to Agentic RAG in which retrieval and generation are incorporated into an agent-based system. Autonomous agents in agentic RAG systems carry out query refinement, corresponding reiterations, validity of relevance and structuring of outputs. This transformation enables the system to have the ability to think about how and when to retrieve information instead of actually conducting retrieval as a single fixed function.

Consequently, agentic RAG systems have better contextual grounding, flexibility and robustness in the more complex knowledge intensive activities like academic research. Besides the review of related work, the chapter also establishes the system requirements and determines the feasibility of making an Agentic RAG-Powered Research Assistant to Academia. The analysis will make sure that the solution proposed is technically feasible, practicable and consistent with the academic research requirements.

2.2 Agentic AI Systems

The agentic AI systems are programmed to act independently and plan the actions, reason about intermediate outcomes, and communicate with other tools or data sources. In contrast to monolithic AI models, which generate the output during a single run, the agent-based systems break down the tasks into smaller goal-oriented steps.

Planning, through which the system identifies a course of action to accomplish a goal, reasoning, through which the system evaluates and refines the output of the intermediate steps, and tool use,

through which the system can interact with APIs, databases or external services, are core attributes of agentic systems. Such skills are especially appropriate in agentic systems where there is a need to make several steps in decision-making and understand what is going on.

The agentic AI systems are particularly useful in the framework of the academic research. Research processes inherently entail several phases namely the purpose of the research, literature search, findings synthesis and source validation. An agent-based model enables these steps to be synchronized so as to be processed and controlled as part of a complex system as opposed to individual tasks. This is why agentic systems are a logical choice when it comes to creating sophisticated research assistants.

2.3 AI-Based Research Assistants

Several AI-based research assistants have been suggested in the last few years, starting with the keyword-enhanced search tools and going up to the AI-based summarization platform. Most systems are directed at single-purpose features like abstract summarization, document clustering or citation extractor. Although these tools can be of helpful aid, they are usually not integrated throughout the entire lifecycle of the research.

One of the weaknesses of the current systems is a weak grounding between retrieved documents and generated outputs. Other tools are able to produce summaries without necessarily limiting the language model to retrieved academic literature, making it more likely to have factual errors. Others are dependent on the keyword matching that restricts their capability of capturing semantic intent.

Such constraints lead to the necessity of the research assistants that combine semantic retrieval, grounded generation, and intelligent orchestration into one system.

2.4 Related Work

The rapid development of Retrieval-Augmented Generation (RAG) and agentic AI systems has opened new directions for research assistance and knowledge discovery. The existing conceptual work, including Zhou et al. [1], focused on the movement toward non-static RAG pipelines to modular and interpretable agentic AI systems, which entails the requirement of planning, execution, knowledge grounding, and tool integration. In addition to this, Singh et al. [2] introduced the survey on Agentic RAG, which is a comprehensive taxonomy with the emphasis on its applications in fields like healthcare, finance, and education. Likewise, the conceptualization of

Agentic RAG put forward by Panda et al. [3] was a scalable and adaptive model, where autonomous learning functions were to be used to enhance reasoning and contextual faithfulness. Collectively, these papers highlight the conceptual basis of agent-based RAG models but also demonstrate that there are no implementations of these models with respect to domains.

Practical advancements have begun to emerge in special situations. Han et al. [4] suggested a platform to automate systematic literature reviews (SLRs) with RAG and demonstrated that it could simplify search, screening, extraction, and synProject Report. The Zhang et al. [5] concept of Agentic Deep Research suggested reasoning based iterative search as being better than keyword-based methods and therefore applied to complex requirements of information. Nagori et al. [6] added an open-source hybrid model of GraphRAG and VectorRAG, which is upgraded by dynamic retrieval and preference optimization, and proves to improve the retrieval accuracy and citation faithfulness significantly. All these practical contributions point to an apparent tendency of hybrid architectures, open-source and reproducible systems.

The recent surveys and prototypes show that agentic RAG is applicable to academic and scientific spheres. Arslan et al. [7] identified an overview of the wider space of RAG methodologies with LLMs, including the summary of evaluation metrics, benchmarks, and future research gaps, but with less emphasis on agentic extensions. Vaykar et al. [8] took a step further to the case of academic application and created an Intelligent Academic Assistant which implemented a course notes and syllabi agentic RAG application. Their prototype obtained high retrieval and faithfulness scores (MRR 0.91, Recall 0.94, Faithfulness 0.95) and gave human feedback of positive improvement, but scalability and data variety are still to be challenged. Taken together, all these studies indicate the potential and shortcomings of existing methods.

Table 2.1: Literature Review

S. No.	Authors	Year	Paper Title	Methodology	Results	Pros	Cons	Limitations
1	Zhou et al.	2025	Towards the Next Generation of Agent Systems: From RAG to Agentic AI	Vision paper analyzing agent modules (planning, execution, knowledge, tool use)	Provides a roadmap for modular, interpretable, next-gen agentic AI systems	Comprehensive analysis of agentic system architecture	No experimental validation	Lacks domain-specific case studies, remains conceptual
2	Singh et al.	2025	Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG	Survey on Agentic RAG, taxonomy, applications, implementation strategies	Defines taxonomy, surveys healthcare/finance/education applications	Detailed taxonomy & real-world applications	Broad scope, lacks deep empirical benchmarks	Limited coverage of open-source implementations
3	Panda et al.	2025	Agentic RAG: Redefining Retrieval-Augmented Generation for	Conceptual framework integrating RAG + autonomous learning	Proposes Agentic RAG as scalable, ethical, adaptive paradigm	Introduces autonomous learning in RAG	Visionary framework, not fully tested	Limited evaluation; mostly theoretical innovation

			Adaptive Intelligence					
4	Han et al.	2024	Automating Systematic Literature Reviews with Retrieval-Augmented Generation	Comprehensive overview + proposed framework for automating SLRs	Framework for automating 4 SLR stages: search, screening, extraction, synProject Report	Practical application to SLR automation	Limited to literature reviews domain	Framework not yet validated at scale; reliance on domain-specific models
5	Zhang et al.	2025	From Web Search towards Agentic Deep Research	Position paper introducing Agentic Deep Research paradigm	Demonstrates superiority of reasoning-driven, iterative RAG for complex search	Strong reasoning-driven search framework	Primarily position-based, limited quantitative results	Focused on web/IR tasks; generalization to academia remains open
6	Nagori et al.	2025	Open-Source Agentic Hybrid RAG Framework for Scientific Literature Review	Built an open-source Python pipeline combining a Neo4j knowledge graph (GraphRAG) and FAISS vector store (VectorRAG).	Instruction-tuned agent + DPO yields sizable metric gains vs baseline (+0.63 VS Context Recall, +0.56 Context Precision, improved faithfulness & precision). Latency analyzed.	Open-source, reproducible Dockerized implementation; dynamic retrieval selection	Some degradation on KG-specific queries; Cypher translation errors;	Evaluation on synthetic benchmarks only; reliance on external APIs; limited multimodal handling
7	Arslan et al.	2024	A Survey on RAG with LLMs	Systematic survey of RAG architectures/applications. Categorizes RAG use cases, retriever/generator designs, evaluation metrics, datasets, and future directions.	Produces a taxonomy of RAG variants, maps applications across domains, summarizes strengths/weaknesses, and suggests research directions (multimodal RAG, robust retrievers).	Broad, clear survey; maps application areas & research gaps; useful references	Review only – no experimental benchmarks; uneven depth across subtopics	Depends heavily on Google Scholar/preprints; may omit paywalled corpora.
8	Vaykar et al.	2025	Intelligent Academic Assistant: An Agentic RAG Framework for QA from Notes and Syllabi.	Implemented prototype: document ingestion (PDF/DOCX/OCR), chunking, embeddings, FAISS vector store, agentic controller.	Strong empirical results: MRR 0.91; Precision 0.88; Recall 0.94; Faithfulness 0.95; ROUGE-1 F1 0.72; SUS 85.5; avg response ≈ 4.5s; 90% task completion; positive feedback.	End-to-end agentic RAG prototype; multimodal ingestion + OCR; thorough evaluation with automated + human metrics; reproducible pipeline	Small dataset (~70 docs); limited disciplines; modest human evaluation sample; UI polishing needed	Scalability not tested; OCR struggles with poor handwriting; no longitudinal studies; dependence on cloud APIs

2.5 Analysis of Existing Academic Search Platforms

The current research support systems mostly refer to the conventional methods of information retrieval. Such popular websites as academic search engines and online libraries are able to access big databases of research papers, articles and reports. Although these systems work well in retrieving documents by use of keywords, it puts a huge load on users to refine their queries by hand, analyse the credibility of the sources and get meaningful information. ArXiv and Semantic Scholar are academic platforms that are instrumental in contemporary research both in terms of open access to preprints in various fields of science and as a principal source of the early dissemination of research. Nevertheless, it facilitates more search by keywords and paper by hand. A majority of the systems which are in place do not automatically generate a summary report and

a structured synProject Report of information. It takes time and results in inconsistent findings as users have to read several documents and combine them on their own. Moreover, citation mining and styling is typically done manually, which adds more chances of mistakes. Even though there are more recent tools that added simplified AI-based summarization functionality, they usually do not rely on any retrieval system, and do not guarantee that the material that is being generated during it is based on credible academic reference materials. This has led to the necessity of more integrated system that would involve the combination of retrieval, summarization, and citation generation into one workflow. This review indicates that current platforms are full of content, but they do not provide much intelligent, end-to-end research support.

2.6 Identified Research Gaps

According to the literature review and analysis of the existing systems, the following gaps in research are identified:

1. **Lack of Intelligent Orchestration:**

   Retrieval, summarization and validation of most systems are not related using an agent-based workflow.

2. **Lack of Grounding of Generated Content:**

   Numerous AI-based summarization tools are not dependable on connecting generated material with the correct academic references.

3. **Limited User Interaction:**

   Current tools are of little help in making interactive exploration, refinement, and questioning of content retrieved.

2.7 Summary of Literature Findings

This chapter helped examine the theoretical background of Retrieval-Augmented Generation, agentic AI systems, and current AI-based research assistants. The comparison shows how the traditional retrieval pipelines have been replaced by agent-based designs that are well suited to multi-step, complex research processes.

The given gaps in the research would create a good rationale as to why the proposed system is supposed to be developed, and it would integrate agentic orchestration and RAG-based semantic retrieval to facilitate the academic research process in a more efficient way. The following chapter is a continuation of these results based on the translation of theoretical knowledge into system requirements and design issues.

CHAPTER 3: SYSTEM REQUIREMENTS & FEASIBILITY ANALYSIS

CHAPTER 3

SYSTEM REQUIREMENTS & FEASIBILITY ANALYSIS

3.1 Functional Requirements

System analysis and design determine the way the proposed application work within itself, and the way various elements interact to bring out the intended results. The proposed Agentic RAG-Powered Research Assistant to Academia is an automated web-based solution that performs automatic opportunities in academic research including information retrieval, summarization, and citation generation.

Table 3.1: Functional Requirements

Requirement ID	Requirement Description
FR-01	The system will enable users to create profiles, log into the system, and manage them.
FR-02	The system will enable users to key in a research topic.
FR-03	The system will be able to access the academic documents of interest in the academic sources.
FR-04	The system will pre-process and store retrieved documents to be semantically searched.
FR-05	The system shall generate structured summaries using a RAG-based approach.
FR-06	The system will retrieve and present citation information that is generated.
FR-07	The system will show the results in a readable and structured format.
FR-08	The system will enable users to export the output generated in academic applications.

3.2 Non-Functional Requirements

Non-functional requirements specify the quality properties of the system and provide its reliability, usability and performance. The system ought to offer tolerable response time of searching, retrieving, and summarizing especially when there is interactive application. Although real-time guarantees are unnecessary, the site should be able to be responsive to near real-time to facilitate academic exploration.

Table 3.2: Non-Functional Requirements

Category	Requirements
Performance	The system must be able to provide first results in a reasonable response time.
Scalability	The system must be able to accommodate a number of users and growing volume of documents.
Security	Secure communications and user data and API key protection should be guaranteed.
Usability	It must be user-friendly and easy to navigate by the academic users.
Reliability	The system must be able to cope with retrieval or processing errors.
Maintainability	The system must be capable of being changed and improved in the future.

3.3 Feasibility Study

The feasibility study determines the practicability and viability of the proposed system in various ways, among them being technical, operational, economical and time feasibility. This analysis is to make sure that the system could be developed, deployed and used in an academic environment with the available resources and within restrictions. The most important points of the feasibility analysis are presented in the table below.

Table 3.3: Types of Feasibility

Type of Feasibility	Description
Technical Feasibility	The system is technically viable because of the maturity and the presence of technologies including Python-based web frameworks, RESTful APIs, vector databases (FAISS and ChromaDB), and pre-trained embedding models. Inference-based large language models accessed through the Groq API are used to implement semantic retrieval, Retrieval-Augmented Generation and do not require any model training or computationally intensive infrastructure.
Operational Feasibility	Operationally the site is operationally viable because it is web-based and can be accessed by the standard browsers without any extra software to be installed. The use of automated research workflow reduces manual labour and the steepness of the learning curve. The modular system architecture allows the components (retrieval, storage and user interface) to be maintained and as well as updated independently.
Economic Feasibility	The system is economical, since it uses open-source libraries as the main source, publicly available academic repositories, and ready-to-use models. It does not need any proprietary data or costly training devices, and running costs are minimal to inference API usage and basic server infrastructure, thus appropriate to the deployment of the system to academic and student levels.
Time Feasibility	The system can be implemented, evaluated and designed in a single academic semester in terms of time feasibility. It is possible to develop rapidly with the help of existing frameworks, APIs, and modular components and have enough time to experiment with them, test, and analyse.

3.4 Chapter Summary

In this chapter, the functional and non-functional requirements of the proposed system were established, and the system was assessed in terms of its feasibility in technical, operational, economic, and time aspects. The review validates that the Agentic-RAG Powered Research Assistant can be applied in reality and implemented within the limitations of an undergraduate Project Report without sacrificing the relevance of the research.

CHAPTER 4: SYSTEM ARCHITECTURE & DESIGN

CHAPTER 4

SYSTEM ARCHITECTURE & DESIGN

4.1 High-Level System Overview

In this chapter, the architectural design and internal structure of the proposed Agentic-RAG Powered Research Assistant to the Academia will be introduced. The system is meant as an agent-driven Retrieval- Augmented Generation (RAG) pipeline which is a modular and web-based application that automates academic literature discovery, semantic retrieval and citation-informed summarization. At a top, the system is client-server based. The user authentication, submitting research query and visualizing the results are all interactive in the frontend. The backend deals with processing of the request, retrieving of the academic sources, indexing of the vectors, and coordination of the reasoning processes based on AI. A specialized AI processing layer combines the use of vector databases and a large language model that is provided through inference APIs to make sure that all of the generated outputs are supported by the retrieved academic content. Focus on structure is modularity, extensibility and transparency. Everything has a clearly defined role, and it can be improved in the future by adding more sources of data, more sophisticated agent coordination, or even some features of joint research without necessarily changing the system design.

4.2 Overall Project Workflow

The total workflow enables the overall execution of the system, beginning with interaction with the user and continuing with the production of structured citation-supported research output. The process starts with the user authentication, thereafter the user enters a research topic using the dashboard. The system filters the input query to enhance the quality of retrieval and retrieves academic documents relevant to the query in arXiv. The documents that have been retrieved are processed, chunked, and stored in a vector database in order to facilitate semantic similarity search. A retrieval and generation agentic RAG controller coordinates retrieval and generation by choosing chunks of documents of interest and feeding them to the language model. The response generated is in the form of a succinct scholarly summary accompanied by a citation metadata. This is the last output that is presented to the user with an option to look at the original paper, to produce summaries or to export results that can be interpreted based on Figure 4.1.

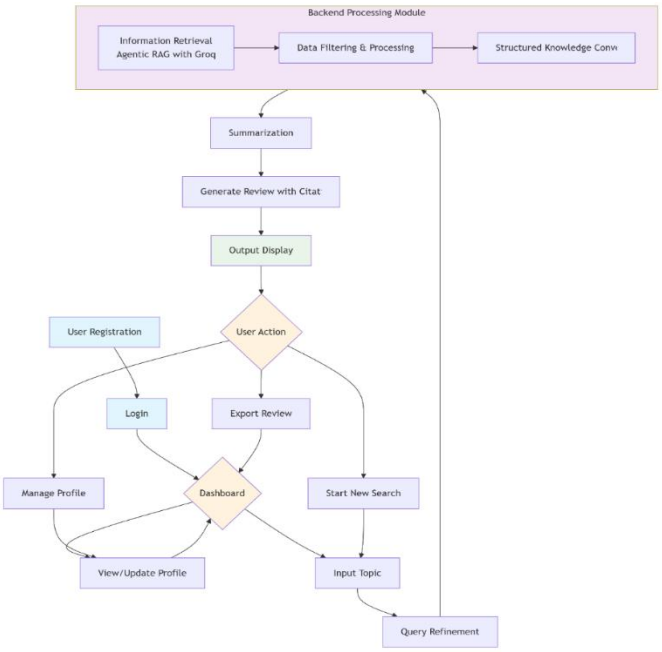

Figure 4.1: Overall Project Workflow Diagram

4.3 System Architecture

System architecture describes the communication between the frontend, the backend and the AI processing elements and data storage layers. The frontend is a single-page application developed with React and is in charge of user interactions, state maintenance, and rendering of results. It provides communication with the backend via RESTful APIs. Authentication, routing of queries and orchestration of retrieval and generation operations is done in the backend which is coded in Django. The AI processing layer combines with an Agentic RAG pipeline, in which the semantic retrieval is provided through vector databases (FAISS and ChromaDB), and the semantic generation is provided by a Groq-powered large language model. External scholarly material is obtained only in arXiv by its public API. This layered architecture brings about separation of concerns and makes the system easier to maintain and extend as shown in Figure 4.2.

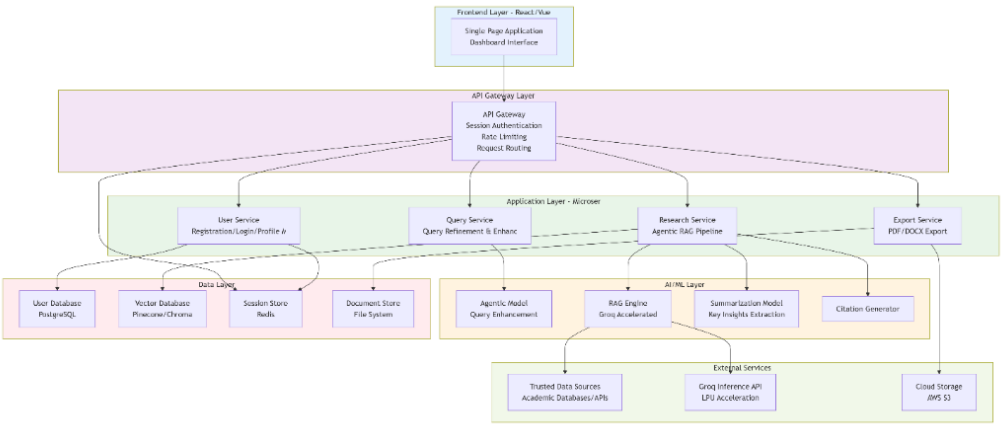

Figure 4.2: System Architecture Diagram

4.4 Module-Wise Design

4.4.1 User Management Module

This module deals with user registration and authentication as well as profile management. It provides the protection of access to system features and retains user-specific data including search history and generated outputs.

4.4.2 Query Refinement Module

Raw user data is processed to eliminate ambiguity to enhance retrieval. This module reorganizes queries into the forms of academic meaning as they are retrieved.

4.4.3 Retrieval Module

The retrieval module retrieves the academic papers in arXiv on the refined query. Metadata like title, authors, abstract, date of publication and PDF links are extracted to undergo further processing.

4.4.4 Vector Store Layer (FAISS & Chroma)

Documents that have been retrieved are transformed to embeddings through a transformer-based embedding model, and stored in FAISS or ChromaDB. The layer supports rapid semantic similarity searches in the course of RAG execution.

4.4.5 Agentic RAG Controller

The agentic controller manages retrieval and generation processes. It does not use a fixed pipeline, but instead chooses the necessary document fragments dynamically and applies tools to make sure that the generated responses are based on evidence that has been retrieved.

4.4.6 Summarization & Citation Module

This module produces formatted summaries based on the language model and an attached citation metadata based on the source documents. The output will be in an academic reading and referencing style.

4.5 Data Flow Design

The Level 0 Data flow diagram (DFD) is a high-level illustration of the flow of data within the system. The input of the users is directed to the backend, and it is retrieved and processed. The documents retrieved are processed by the vector database and AI processing modules and structured results are then returned to the front end where it is properly exhibited in the Figure 4.3.

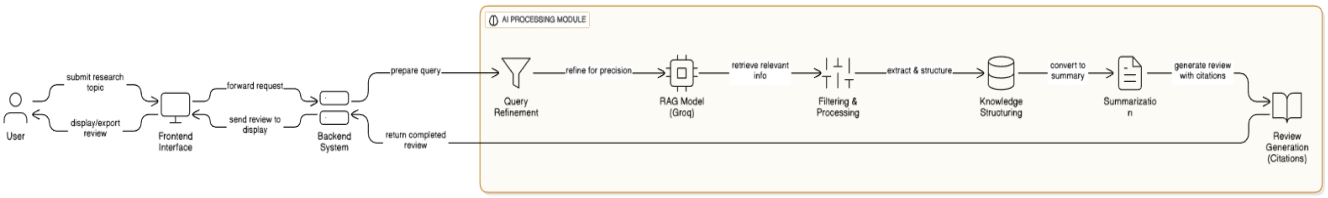

Figure 4.3: Level 0 Data Flow Diagram (DFD)

4.6 UML Diagrams

4.6.1 Sequence Diagram

The sequence diagram shows how user, frontend, backend, vector store and AI processor interact in an effort to respond to a research query. It brings out request propagation, retrieval, RAG processing, and delivery of responses which is evidently illustrated in Figure 4.4.

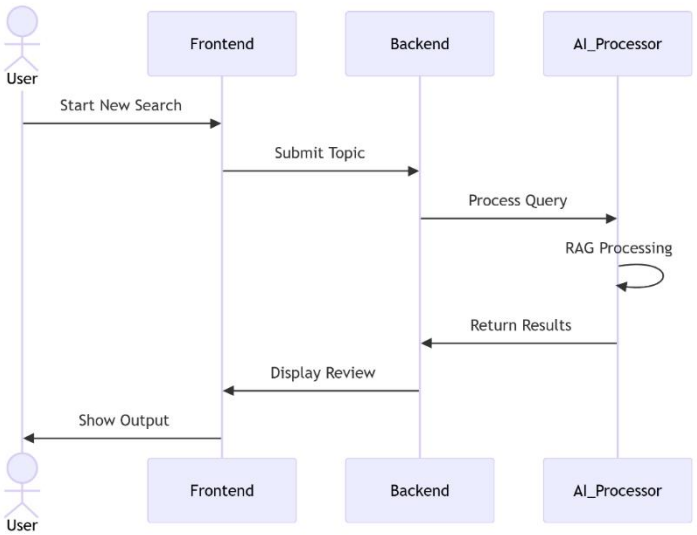

Figure 4.4: Sequence Diagram

4.6.2 Activity Diagram

This is indicated in the activity diagram which shows the logical sequence of user and system actions, starting with the login and submission of a topic to ending with the creation of results and their export, Figure 4.5 makes this clear.

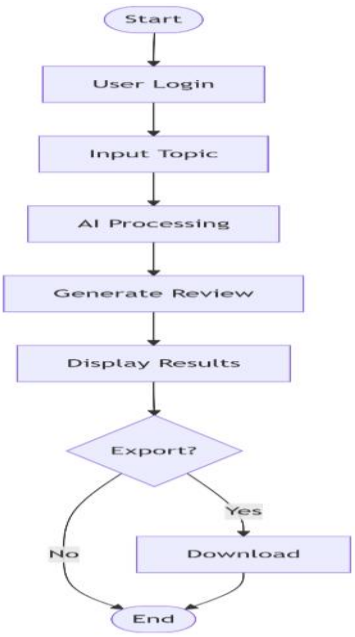

Figure 4.5: Activity Diagram

4.7 Deployment Architecture

The deployment architecture explains hosting and execution of system components. Frontend is an environment based on a browser, whereas the backend and AI services are provided in a server. The deployment of vector databases is in the form of a local or a server-side service to provide low-latency access. This can be deployed flexibly in the academic environment, whether as local servers or cloud-based infrastructure Figure 4.6 illustrates the same.

.

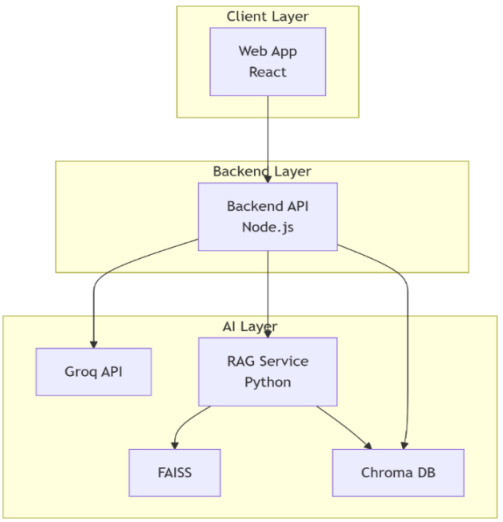

Figure 4.6: Deployment Architecture Diagram

4.8 Design Decisions and Trade-offs

A number of design choices were undertaken to trade system capability, complexity and feasibility:

- **Inference over Training:** Inference APIs can be used to run pre-trained models, eliminating the computational cost and complexity of training custom models.
- **Single Data Source:** arXiv has been chosen due to its reliability and openness at the expense of a wider coverage and academic trust.
- **Agentic Control vs Static Pipelines:** An agentic RAG was selected to enhance grounding and flexibility but with the cost of complexity of the system.
- **Vector Databases:** FAISS and ChromaDB were chosen because they are able to support semantic retrieval efficiently, focusing not on basic keyword search but on speed and relevance.

CHAPTER 5: IMPLEMENTATION DETAILS & RESULTS

CHAPTER 5

IMPLEMENTATION DETAILS & RESULTS

5.1 Introduction

The chapter shows the details of implementation and the outcomes of Development of an Agentic RAG-Powered Research Assistant for Academia. This chapter is aimed at showing how the system works based on interfaces implemented and generated outputs. As opposed to theoretical assessment, this chapter clearly brings out practical behaviour of the system through real execution screenshots taken during the development.

The findings comprise user authentication, dashboard interaction, paper search in academics, AI-assisted generation of summaries, and the results in various fields of research. These deliverables confirm that the system is working as per the plan and achieves the goals that were outlined at the beginning of the report.

5.2 Technological Stack Overview

The deployment of the "Agentic-RAG Powered Research Assistant in Academia" is founded on a current web and artificial intelligence stack that is meant to support customizable development, standardized retrieval, and inference.

- **Frontend:** The frontend is in React.js in a component-based architecture. React was selected due to its flexibility, responsiveness, and appropriateness in the programming of one-page academic websites. It manages user authentication, research query, visualization of results, and connection to the backend APIs based on HTTP requests.

- **Backend:** Django is used as the fundamental application server that is built. Django handles user authentication, routing of the API, validation of requests and coordinates the research pipeline. The REST-style endpoints are necessary to facilitate the flow of communication between the frontend and the backend components.

- **AI / ML Stack:** The AI layer is developed based on LangChain, FAISS, ChromaDB, and Groq inference API. Rather than training custom models, the system uses pre-trained

embedding models and large language models that are accessed via inference and guarantee quick response time and less computational baggage.

5.3 Backend Implementation

- **Django API Design**: Django API Design: The back-end has several API endpoints that perform the following tasks; authentication, search of research, answer questions, and profile management. These endpoints are accomplished with Django views and are based on a request-response model. The Key backend capabilities are:
  1. User registration and login
  2. Query submission and parameter handling
  3. Triggering retrieval and RAG processing
  4. Returning structured JSON responses to the frontend

- **arXiv Integration:** It uses the arXiv Python API to perform academic data search. On entering a research query, the backend creates a search request with parameters including key words, number of results, date range and sorting option. Metadata such as title, authors, abstract, publication date, PDF URL and DOI (where applicable) are extracted and encoded into a structured format on each of the retrieved papers. This information is the basis of downstream semantic processing**.**

- **RAG Pipeline Triggering:** When data retrieval is finished, then the backend starts the RAG pipeline. Retrieved papers are created as document objects and fed into the vectorization and retrieval modules and then agentic thinking is invoked.

5.4 Agentic RAG Pipeline Implementation

- **Embeddings:** Embeddings Document embeddings are obtained with the Hugging Face ecosystem transformer-based sentence embedding model. The documents are transformed into dense vectors which encode semantic information instead of surface-level keywords. Such embeddings allow similarity-based retrievable questions during summarization and answering.

- **Retrieval:** Vector similarity search is used to execute semantic retrieval. When a query is made the most relevant document chunks are retrieved using cosine similarity. This makes sure that the generation model does not run on information that is not relevant to academics and that does not fit the context.

- **Groq LLM Integration:** The language model is made available through the Groq inference API that offers high performance execution of large language models. The LLM is only applied to inference and generation of content, and outputs are not limited to be detached to documents retrieved.

- **Tool-Based Agents:** LangChain tool-based agents are used as an agentic architecture. The agent guides the search and reasoning processes of documents through the dynamic invocation of retrieval tools. This method permits adaptable orchestration of retrieval and generation enhancing grounding and lessening hallucination.

5.5 Vector Database Implementation

- **FAISS:** It serves as the main semantic search in-memory store. It allows rapid similarity search and is also ideal in dense embeddings when performing real-time research queries. FAISS indices are also stored locally so that they can be reused between sessions.

- **ChromaDB:** It can be used optionally in persistent storage of vectors. It provides metadata in document storage and enables retrieval of larger datasets on a scale. Switching between FAISS and ChromaDB is possible without altering the retrieval logic because of the system design. Both have a common interface used to access all functionality in which a store of vectors can be retrieved, and makes it easier to maintain and extend.

5.6 Frontend Implementation

- **React Components:** The React-based frontend contains reusable react components and reacts to clean separation of concern and easy maintenance of the UI, such as:
  1. Login and authentication views
  2. Dashboard and search interface

3. Research paper listing and detail views
4. AI summary and citation display components

- **Authentication Flow:** User authentication is performed under a simple flow and the back end authenticates the credential and returns a session identifier. It is known as an identifier to access user-specific data and authorize control to secured routes of the application.

- **Search and Result User Interface:** The search interface enables the user to type the research topics, filter it and start the retrieval. The findings are presented in a tabular format with paper metadata, abstracts, and the options to access PDFs, generate AI summaries, or pose follow-up questions. The UI design is also focused on usability and clarity such that the academic user will not need to be concerned with the complexity of the system, as they will be able to concentrate on the content.

5.7 User Authentication and Access Interface

The system will start with a secure user authentication interface through which the registered users can log in and access the research platform. This module ensures that the system has a restricted access to the system and only authorized users can interact with the system and use its research assistance capabilities. The home page includes the input fields of institutional email and password, and user registration. After authentication, the user will get redirected to the main dashboard of the app. The interface defines the initial contact of the user and the system and provides access to the research process under control.

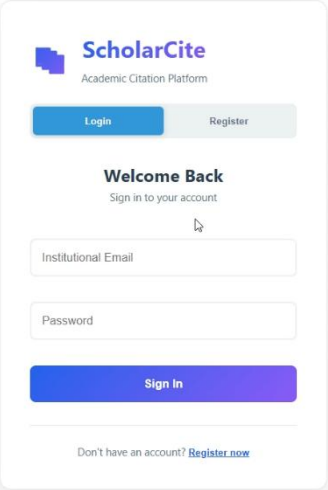

Figure 5.1: Clean login page for ScholarCite platform with email and password fields.

5.8 Dashboard and Research Search Interface

Once a user has successfully logged in, he or she is greeted with the primary dashboard of the platform. The dashboard is the main place where users can start research queries, use filters and search academic papers. The dashboard has a research search bar, navigation controls, and points off features that represent the main functions of the dashboard like fast search, AI reviews, and clever filtering.

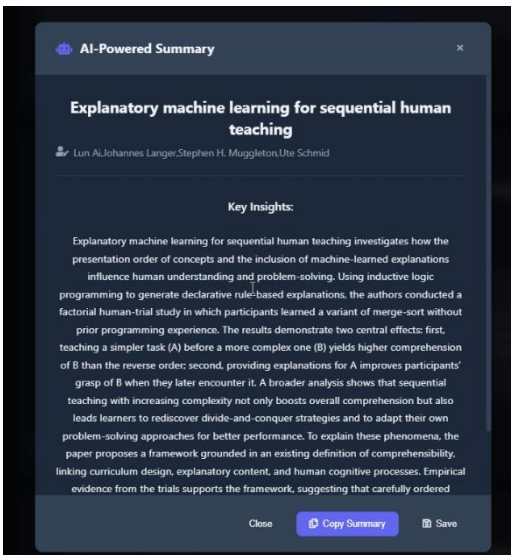

Figure 5.2: ScholarCite dashboard with research paper search bar and

Besides the dashboard overview, the system offers an organized research discovery interface within which the user is able to search papers by use of the keywords, titles or subject areas. Search filters like the year of publication and the number of results enable the user to filter the search results to obtain more relevant results. It is an interface that allows to explore academic content in an efficient way and serves as the starting point of further AI-based processing.

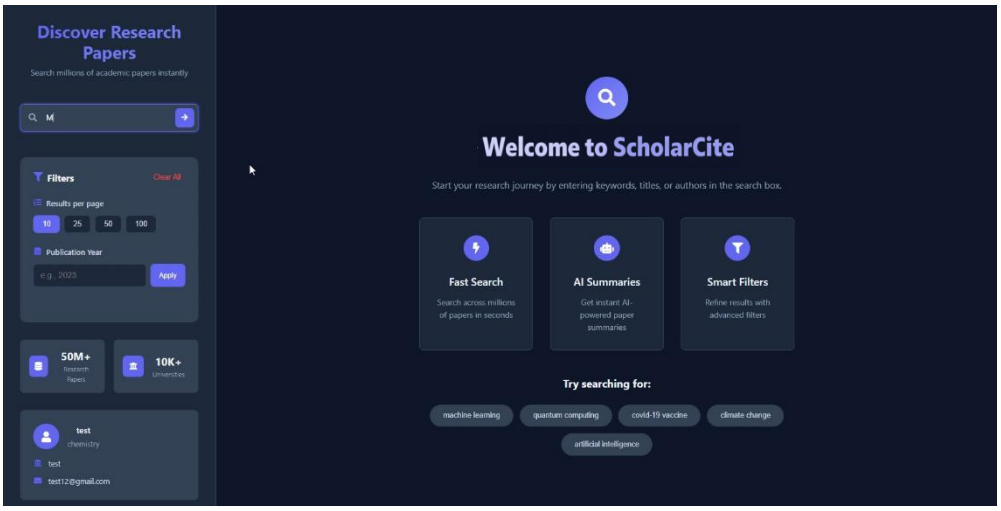

Figure 5.3: Academic research platform with search filters and paper database

5.9 Research Paper Retrieval and Viewing

The system searches and retrieves relevant academic papers on related scholarly sources once search query is entered. The retrieved papers are presented with the title, authors, publication information, and abstract of each paper that enables the user to evaluate relevance of the paper speedily. It also facilitates the access of full research papers redirecting the user to the original PDF of the article on the publisher or repository site. This will guarantee transparency and maintain academic authenticity and this will enable the users to get a summary of the insights and the entire original documents when needed.

.

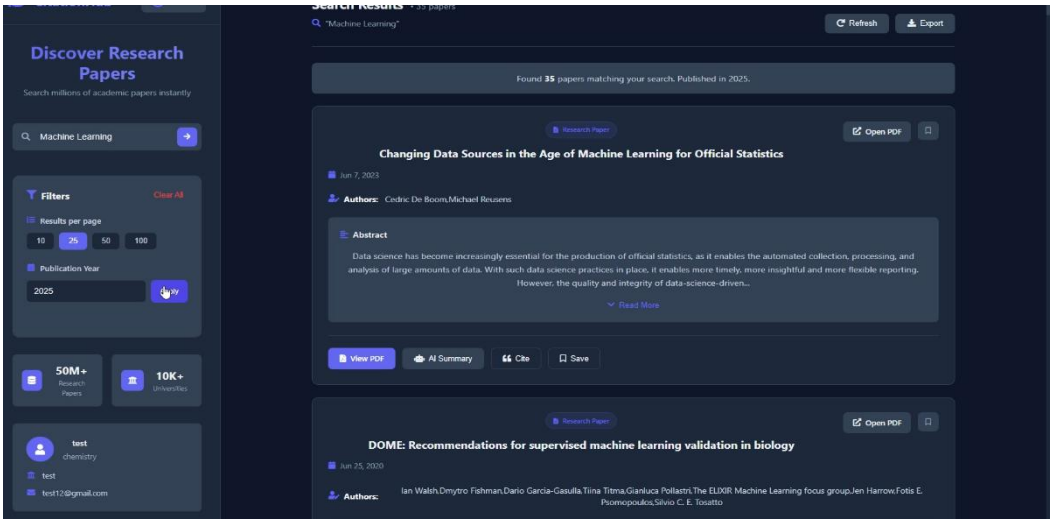

Figure 5.4: Research paper summary page with abstract and AI-generated insights

Machine learning manuscript No.
(will be inserted by the editor)

Explanatory machine learning for sequential human teaching

Lun Ai · Johannes Langer · Stephen H. Muggleton · Ute Schmid

Abstract The topic of comprehensibility of machine-learned theories has recently drawn increasing attention. Inductive Logic Programming (ILP) uses logic programming to derive logic theories from small data based on abduction and induction techniques. Learned theories are represented in the form of rules as declarative descriptions of obtained knowledge. In earlier work, the authors provided the first evidence of a measurable increase in human comprehension based on machine-learned logic rules for simple classification tasks. In a later study, it was found that the presentation of machine-learned explanations to humans can produce both beneficial and harmful effects in the context of game learning. We continue our investigation of comprehensibility by examining the effects of the ordering of concept presentations on human comprehension. In this work, we examine the explanatory effects of curriculum order and the presence of machine-learned explanations for sequential problem-solving. We show that 1) there exist tasks A and B such that learning A before learning B results in better comprehension for humans in comparison to learning B before learning A and 2) there exist tasks A and B such that the presence of explanations when learning A contributes to improved human comprehension when subsequently learning B. We propose a framework for the effects of sequential teaching on comprehension based on an existing definition of comprehensibility and provide evidence for support from data collected in human trials. Our empirical study involves curricula that teach novices the merge sort algorithm. Our results show that sequential teaching of concepts with increasing complexity a) has a beneficial effect on human comprehension and b) leads to human re-discovery of divide-and-conquer problem-solving strategies, and c) allows adaptations of human problem-solving strategy with better performance when machine-learned explanations are also presented.

Keywords Explainable artificial intelligence; Machine learning comprehensibility; Meta-interpretive learning; Inductive logic programming.

Lun Ai ✉
Department of Computing, Imperial College London, London, UK
E-mail: lun.ai15@imperial.ac.uk

Johannes Langer
University of Bamberg, Bamberg, Germany
E-mail: johannes-niran.langer@stud.uni-bamberg.de

Stephen H. Muggleton
Department of Computing, Imperial College London, London, UK
E-mail: s.muggleton@imperial.ac.uk

Ute Schmid
Cognitive Systems Group, University of Bamberg, Bamberg, Germany
E-mail: ute.schmid@uni-bamberg.de

arXiv:2205.10250v2 [cs.AI] 26 Mar 2023

Figure 5.5: Redirection to original research paper PDF source

5.10 AI-Based Summary Generation

One of the most important characteristics of the system is the possibility to produce some AI-driven summaries of a chosen research paper. Upon the user triggering the process of summary generation, the paper content is processed in the Retrieval- Augmented Generation pipeline by the system. The created summary will contain the most important points in an organized and readable form that will allow users to realize the main contributions of the paper without reading the entire work. The generated summary is given options to copy or save the summary to be used later and this feature greatly saves time taken to read the summary manually and promotes effective knowledge acquisition.

.

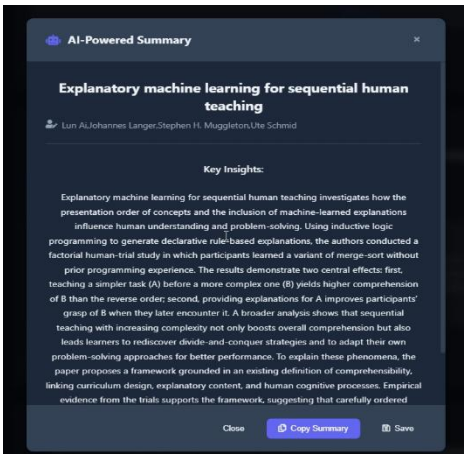

Figure 5.6: AI summary generation page for academic research paper

5.11 Output Results for Different Search Domains

The searches were conducted in various academic fields to prove the flexibility of the system. The system was able to retrieve the pertinent papers and produce orderly outputs regardless of the area of research. Below is an example of search results based on the biological sphere, which demonstrates that the system can deal with diversity across the domain and can ensure the same level of output and such results indicate that the system is not specific to the particular discipline and can facilitate multidisciplinary research.

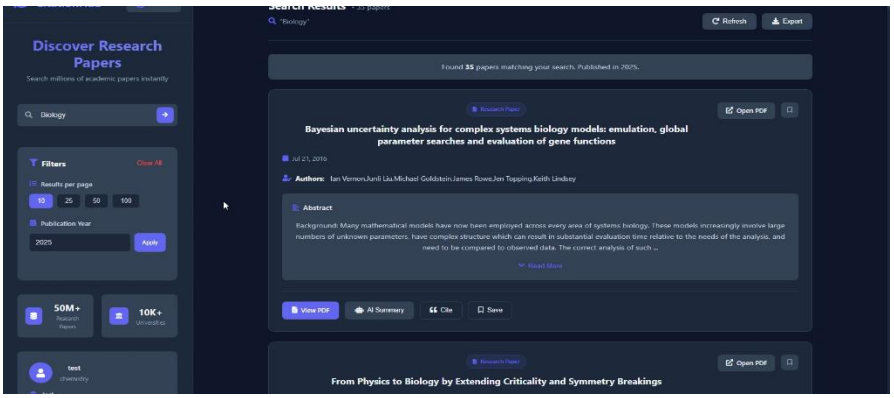

Figure 5.7: Output results for biology-based academic search

5.12 Summary of Results

The results of the implementation indicate that the system manages to carry out user authentication, academic paper retrieval, AI-based summarization, and structured output generation. The interfaces are user-friendly, and the results obtained are applicable and organized. On the whole, the system serves its purpose of helping users in their academic research because it automates the retrieval and summarization of information and offers access to original sources. The findings confirm the suitability and usefulness of the solution proposed.

CHAPTER 6: DISCUSSION

CHAPTER 6

DISCUSSION

6.1 System Strengths

The proposed Agentic RAG-Powered Research Assistant has strengths that qualify it to be used in academic research assistance. The agentic architecture that allows an organized coordination among the query refinement, retrieval, semantic matching and generation is one of the greatest strengths of the system. The agentic design is not applied as the tool, unlike traditional AI-based tools, considers each step separately, but the stage adds value to the final output. The other strength is the semantic retrieval which has been used with the help of the vector databases. Using embeddings and search through similarity, the system finds the research papers according to the relevance to the context and not the mere coincidence of the keywords. This will enhance the quality of the retrieved documents and it is much closer to the actual practice of academic research in the real world.

Grounded generation is also a focus of the system in which summaries are generated solely based on retrieved academic material. Such a design will decrease the chances of hallucinations and increase the credibility. Also, the Groq API has allowed the incorporation of high-speed inference to generate near real-time responses and thus the platform can be used in interactive mode. Lastly, both the back and front end are modularly designed which enables easy future extension hence making the system a formidable research and development platform.

6.2 Limitations of the Current System

Despite its strengths, the existing system has a number of weaknesses that must be mentioned. To begin with, the system uses the academic data source in the form of arXiv only. Although it is a source of quality preprints of numerous fields, not all journals, conferences as well as peer-reviewed publications are available in arXiv. This restricts the scope of the literature that the users can access.

Second, there is no fine-tuning or training of the custom models in the system. All reasoning and summarization associated with AI is executed with the help of pre-trained large language models and inference-only APIs. Although this method of development makes the development process

easier and provides the system with stability, it restricts the system to accommodate domain-specific writing styles or disciplines.

Also, the system at the moment specializes in text-based research papers. Multimodal contents like figures, tables, mathematical notations and scanned documents are not interpreted past the textual representations. There is also a lack of large-scale quantitative benchmarking of the system and the system is mainly based on functional validation and qualitative observations. These constraints point to possibilities of further development as opposed to design shortcomings.

6.3 Ethical and Academic Integrity Considerations

The most important consideration that any AI-assisted research system must have is ethical use and academic integrity. The system suggested is created to assist the researchers without substituting the critical thinking or creative academic input. Every summary, which was generated, is clearly based on retrieved scholarly works, and the direct access to original research articles is provided to the users.

The system eliminates the production of unsupported claims by only allowing the context of the language model to be restricted to documents that have been retrieved. Metadata of citation is maintained in order to promote proper citation and accountable academic applications. Notably, the platform does not create portrayed content as research, but rather as aid toward insight and discovery. In terms of data ethics, the system only gets access to publicly accessible academic data via official APIs, without storing restricted or copyrighted full-text records in excess of the allowed usage. The user data comprises of basic authentication details, and is stored safely. On balance, the system can be considered as compliant with the principles of responsible AI because it fosters transparency, attribution, and ethical academic assistance instead of replacing the content.

CHAPTER 7: CONCLUSION

CHAPTER 7

CONCLUSION

5.1 Conclusion

This Project Report included the design and implementation of an Agentic-RAG Powered Research Assistant to Academia to enhance the process of finding, comprehending and utilizing academic literature. This work was driven by the need to resolve the increasing predicament of students and researchers in light of information overload, disjointed research processes, and manual literature review processes.

The semantic retrieval, vector databases and large language model inference are combined into an agentic Retrieval-Augmented Generation framework in the proposed system. With the system basing the generation on retrieved academic documents, the system makes sure that the output is up to date, traceable and consistent with scholarly sources. Embedding-based retrieval and high-speed inference allow offering effective and context-sensitive research support with the use of arXiv as a trusted academic repository.

The systematic implementation and evaluation allowed the system to show that it could consistently retrieve the pertinent research articles, create coherent summaries, and facilitate the exploration of research in a variety of academic areas. The clear separation of concerns and modular architecture, combined with the inference-based AI design, make the system not only practical but also applicable to the actual world but also a viable basis of the subsequent academic research. On balance, this paper proves that agentic RAG-based systems are capable of benefiting the process of academic research when grounding, transparency, and usability are the key principles to consider.

5.2 Future Scope

The existing system offers an efficient and academically-based research assistant, one can consider a number of extensions to make it more powerful and research-worthy:

1. **Customization of Citation Style:** It can be customized by supporting various citation styles including IEEE, APA and MLA as necessary for various academic and publication requirements.

2. **Multi-Agent Planning and Reasoning:** Future versions can be modified to add more specific agents, namely query planning, evidence validation, cross-document reasoning, and result refinement.
3. **Integration of Additional Academic Sources:** To increase coverage and diversity of literature, it would be better to expand the list of additional academic sources to include arXiv, Semantic Scholar, CrossRef, and institutional databases.
4. **Formal Evaluation and Benchmarking:** Quantitative measures like retrieval precision, recall, MRR, faithfulness and human evaluation experiments can be used to assess the system to further empirically validate it.
5. **Collaborative Research Support:** The system can be multi-user as a research workspace, with support features, including the ability to share workspaces, add notes, and collaboratively review literature.

REFERENCES

[1] Yingli Zhou and Shu Wang. Towards the Next Generation of Agent Systems: From RAG to Agentic AI. VLDB 2025 Workshop: LLM+Graph.

[2] AGENTIC RETRIEVAL-AUGMENTED GENERATION: A SURVEY ON AGENTIC RAG https://doi.org/10.48550/arXiv.2501.09136

[3] Panda, Meethun. (2025). Agentic RAG Redefining Retrieval-Augmented Generation for Adaptive Intelligence.

[4] Han, B., Susnjak, T., & Mathrani, A. (2024). Automating systematic literature reviews with retrieval-augmented generation: A comprehensive overview. *Applied Sciences*, *14*(19), 9103. https://doi.org/10.3390/app14199103

[5] From Web Search towards Agentic Deep Research: Incentivizing Search with Reasoning Agents. https://doi.org/10.48550/arXiv.2506.18959

[6] Open-Source Agentic Hybrid RAG Framework for Scientific Literature Review https://doi.org/10.48550/arXiv.2508.05660

[7] Arslan, M., Ghanem, H., Munawar, S., & Cruz, C. (2024). A survey on rag with llms. *Procedia Computer Science*, *246*, 3781–3790. https://doi.org/10.1016/j.procs.2024.09.178

[8] Yash Anil, & Dr. MADHAVI. (2025). Intelligent academic assistant: An agentic RAG framework for QA from notes and Syllabi. *International Journal on Science and Technology*, *16*(2). https://doi.org/10.71097/ijsat.v16.i2.5300

ANNEXURE - I

Sample System Outputs

This section illustrates typical outputs generated by the system during execution.

Retrieved Paper Metadata

- Title
- Authors
- Publication Date
- Abstract
- PDF URL
- DOI (when available)

AI-Generated Summary

- Concise paragraph-based explanation
- Grounded strictly in retrieved documents
- Designed for academic readability

User Query Response

- Contextual answer generated through agentic RAG
- Based on semantic retrieval and LLM inference

ANNEXURE – II

Deployment Notes

The system is deployed as a modular web application with separate frontend and backend services.

Backend Deployment

- Framework: Django
- Environment: Conda-based Python environment
- Vector Store: FAISS (local) / ChromaDB (optional persistence)
- External API: arXiv
- LLM Inference: Groq API

Frontend Deployment

- Framework: React.js
- Runs independently on a local development server
- Communicates with backend via REST APIs

Execution Environment

- IDE: Visual Studio Code
- OS: Windows / Linux / macOS
- Browser: Chrome / Firefox